kubectl logs in Kubernetes

Introduction

kubectl logs is a **Kubernetes command** used to **retrieve logs from a running Pod**.

- **What does kubectl logs do?**
- **Why is kubectl logs important?**
- **How can we use it to troubleshoot applications?**

This guide will help you **understand and use** kubectl logs effectively.

Section 1: Learn

What is kubectl logs?

kubectl logs fetches **standard output (stdout) and error logs (stderr)** from a running Pod to help **debug and monitor applications**.

Feature	Description
Retrieves Logs	Fetches logs from a running Pod
Monitors Application Status	Helps debug errors and issues
Works on Multi-Container Pods	Retrieves logs from specific containers
Supports Streaming Logs	Provides real-time log monitoring

Why Use kubectl logs?

1. **Debugging** – Identify why a Pod is failing.
2. **Performance Monitoring** – Check logs for slow requests or errors.
3. **Real-Time Troubleshooting** – Stream logs as they are generated.
4. **Event Tracking** – Detect crashes, memory issues, or unexpected behavior.

Interesting Fact

If a Pod has multiple containers, kubectl logs requires specifying the container name to fetch logs correctly.

Section 2: Practice

1. Basic kubectl logs Commands

Get Logs of a Single Pod

```
kubectl logs <pod-name>
```

Get Logs of a Specific Container Inside a Pod

```
kubectl logs <pod-name> -c <container-name>
```

Stream Live Logs

```
kubectl logs -f <pod-name>
```

Get Logs from a Previous Pod Instance

```
kubectl logs <pod-name> --previous
```

2. Filtering and Debugging Logs

Show Only the Last 100 Log Lines

```
kubectl logs <pod-name> --tail=100
```

Get Logs from All Pods in a Deployment

```
kubectl logs -l app=my-app
```

Save Logs to a File for Analysis

```
kubectl logs <pod-name> > logs.txt
```

Watch Logs with a Timestamp

```
kubectl logs <pod-name> --timestamps
```

3. Debugging Common Issues with kubectl logs

If Pod is Not Running

```
kubectl get pods

kubectl describe pod <pod-name>
```

Check System Events for Issues

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Section 3: Know More

Frequently Asked Questions

1. **What if the Pod is not running?**
   - kubectl logs works only for running Pods. Check status with kubectl get pods.
2. **How do I check logs from a crashed Pod?**
   - Use kubectl logs <pod-name> --previous.
3. **Can I filter logs for specific keywords?**
   - Yes, use kubectl logs <pod-name> | grep "error".
4. **How do I get logs from a Pod with multiple containers?**
   - Specify the container name:

```
kubectl logs <pod-name> -c <container-name>
```

5. **How do I continuously watch logs?**
   - Use kubectl logs -f <pod-name>.

Conclusion

kubectl logs is **one of the most powerful Kubernetes commands** for **monitoring and debugging applications**.

It allows developers to **troubleshoot issues efficiently** and ensure smooth application performance.